Numerical Linear Algebra in Python

A compact educational project that implements classical numerical linear algebra algorithms in Python using NumPy — including LU and QR decompositions, Gaussian elimination with pivoting, iterative methods (Jacobi and Gauss–Seidel).

Implemented topics

- **LU decomposition** with partial or full pivoting
- **Determinant**, **inverse matrix**, and **condition number** via LU
- **Gaussian elimination** for rank, consistency, and singular systems
- **QR decomposition** via Householder reflections
- **Jacobi** and **Gauss–Seidel** iterative methods
- Numerical checks: residuals, reconstruction errors, and convergence history

Mathematical background

LU decomposition and solving linear systems (lu.py)

Core idea

The matrix is factorized into triangular factors:

$$PAQ = LU$$

where:

- L is lower triangular,
- U is upper triangular,
- P and Q are permutation matrices.

The purpose of pivoting is to move a numerically stable pivot to the diagonal. Row permutations are stored in P, and column permutations are stored in Q.

How the decomposition is built

At step k, the algorithm looks at the submatrix $U_{k:n,\,k:n}$ and chooses a pivot element.

With full pivoting, the largest element in the remaining submatrix is selected: $\max_{i,j} |u_{ij}|$

Then:

1. the chosen row is swapped with row k,
2. the chosen column is swapped with column k,
3. elimination is performed below the pivot.

The multipliers are stored in L:
$$l_{ik} = \frac{u_{ik}}{u_{kk}}, \qquad i > k$$
and the trailing part of the matrix is updated by
$$u_{ij} \leftarrow u_{ij} - l_{ik} u_{kj}$$

This is the standard Gaussian elimination mechanism written in factorized form.

Determinant and inverse

From the factorization,
$$\det(A) = \frac{\det(U)}{\det(P)\det(Q)}$$

since $\det(L) = 1$ for unit lower triangular matrices.

The inverse is built column by column:
$$Ax_i = e_i$$

so that
$$A^{-1} = [x_1, \ldots, x_n]$$

Condition number

The infinity-norm condition number is computed as
$$\kappa_\infty(A) = \|A\|_\infty \|A^{-1}\|_\infty$$

It measures how sensitive the solution is to perturbations in the data.

QR decomposition and solving linear systems (qr.py)

Core idea

The matrix is factorized as
$$A = QR$$

where:

- Q is orthogonal,
- R is upper triangular.

The orthogonality condition is
$Q^\top Q = I$

This makes QR decomposition especially stable numerically.

How the decomposition is built

The implementation uses Householder reflections.

At step k, take the vector formed by the part of the current column below the diagonal:
$x = R_{k:n,\,k}$

Construct the reflector vector
$v = x + \operatorname{sgn}(x_1)\|x\|_2 e_1$

and then define the Householder matrix
$$H = I - 2\frac{vv^\top}{v^\top v}$$

This matrix is orthogonal and symmetric. Left-multiplying by H zeros all entries below the first component of x.

The algorithm applies these reflectors sequentially to the trailing submatrix, so after all steps the matrix becomes upper triangular:
$R = H_m \cdots H_2 H_1 A$

At the same time, the orthogonal factor is accumulated as a product of reflections:
$Q = H_1 H_2 \cdots H_m$

Since each H_i is orthogonal, the product Q is orthogonal as well.

Iterative methods (iterative_methods.py)

Core idea

Instead of solving $Ax = b$ directly, iterative methods construct a sequence
$x^{(0)}, x^{(1)}, x^{(2)}, \ldots$

that converges to the exact solution x^*.

The generic stationary form is
$x^{(k+1)} = Bx^{(k)} + c$

Convergence is guaranteed when the iteration matrix is contractive, for example when
$\|B\|_\infty < 1$

or, more generally, when
$\rho(B) < 1$

where $\rho(B)$ is the spectral radius.

Matrix splitting

The matrix is decomposed as
$A = D + L + U$
where:

- D is diagonal,
- L is strictly lower triangular,
- U is strictly upper triangular.

This splitting is the basis for Jacobi and Gauss–Seidel methods.

Jacobi method

Jacobi updates every component using only values from the previous iteration:
$$x_i^{(k+1)} = \frac{b_i - \sum_{j \neq i} a_{ij} x_j^{(k)}}{a_{ii}}$$
In matrix form:
$x^{(k+1)} = B_J x^{(k)} + c_J$
with
$B_J = -D^{-1}(L + U)$
This method is simple and fully parallelizable, because all coordinates use only the previous vector $x^{(k)}$.

Gauss–Seidel method

Gauss–Seidel improves Jacobi by using newly computed components immediately:
$$x_i^{(k+1)} = \frac{b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)}}{a_{ii}}$$
In matrix form:
$B_{GS} = -(D + L)^{-1} U$
This usually converges faster than Jacobi because each update uses the most recent available information.

Diagonal dominance

The helper
$is_diagonally_dominant$
checks the sufficient condition

checks the sufficient condition

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|$$

(or the non-strict version). Diagonal dominance is a classical condition that often guarantees convergence of stationary iterations.

Stopping criterion

The methods stop when the infinity norm of the difference between successive iterates becomes small:

$$\|x^{(k+1)} - x^{(k)}\|_\infty < \varepsilon$$

The code also stores the history of these differences.

Error estimates

A priori estimate:

$$k \geq \frac{\log(\varepsilon(1-q)/\|x^{(1)} - x^{(0)}\|_\infty)}{\log q}$$

A posteriori estimate:

$$\|x^* - x_k\|_\infty \leq \frac{q}{1-q}\|x_k - x_{k-1}\|_\infty$$

These formulas quantify how many iterations are needed and how accurate the current approximation is.

Gaussian elimination and singular systems (linear_system.py)

Core idea

Gaussian elimination transforms the system matrix into echelon form by eliminating entries below each pivot. The algorithm works row by row and uses pivoting to avoid division by very small numbers.

The resulting form is upper echelon:

$$U$$

and the elimination can be written schematically as

$$PAQ = U$$

for full pivoting, or as a row-pivoted version when only row swaps are used.

How elimination works

At step k:

1. choose a pivot in the current column or current submatrix,
2. swap rows so that the pivot goes to position (k, k),

3. if full pivoting is enabled, swap columns too,
4. eliminate all entries below the pivot.

The elimination factor for row i is
$$m_{ik} = \frac{u_{ik}}{u_{kk}}$$
and the row update is
$$r_i \leftarrow r_i - m_{ik} r_k$$

Rank

The rank is the number of nonzero rows in echelon form:
$$\mathrm{rank}(A)$$
A matrix is degenerate if
$$\mathrm{rank}(A) < \min(n, m)$$

Consistency

A linear system is inconsistent if elimination produces a row of the form
$$0 = c, \qquad c \neq 0$$
which means the equations contradict each other.

Solving singular systems

For a consistent singular system, the code finds one particular solution.

After elimination, pivot positions are identified, and back substitution is applied only on pivot variables. Free variables are set to zero. Since column permutations may have been used, the solution is then mapped back to the original variable order.

So the final result is one valid solution of
$$Ax = b$$
not necessarily the unique solution.

Small utility functions (utils.py)

These helpers support the main algorithms and implement basic linear algebra operations.

Vector norms	Matrix norms	Residual
$\|x\|_\infty = \max_i \|x_i\|$ $\|x\|_1 = \sum_i \|x_i\|$ $\|x\|_2 = \sqrt{\sum x_i^2}$	$\|A\|_\infty = \max_i \sum_j \|a_{ij}\|$ $\|A\|_1 = \max_j \sum_i \|a_{ij}\|$ $\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2}$	Measures how well the approximate solution satisfies the system: $r = Ax - b$ $\|r\|_\infty = \|Ax - b\|_\infty$

Forward substitution (lower triangular system):

$Lz = b$

$$z_i = \frac{b_i - \sum_{j=1}^{i-1} l_{ij} z_j}{l_{ii}}$$

Backward substitution (upper triangular system):

$Ux = b$

$$x_i = \frac{b_i - \sum_{j=i+1}^{n} u_{ij} x_j}{u_{ii}}$$

Repository structure

```
numerical-linear-algebra/
│
├── src/
│   ├── iterative_methods.py
│   ├── linear_systems.py
│   ├── lu.py
│   ├── qr.py
│   ├── utils.py
│
├── tests/
│   ├── test_lu.py
│   ├── test_qr.py
│   ├── test_iterative.py
│
├── examples/
│   ├── demo_iterative.py
│   ├── demo_linear_system
│   ├── demo_lu.py
│   ├── demo_qr.py
│
├── main.py
├── README.pdf
├── LICENSE
```

Why this project is useful

I wrote this to understand how numerical linear algebra works. The goal of the project is to study LU and QR decompositions, Gaussian elimination with pivoting, iterative methods (Jacobi and Gauss–Seidel), and the effect of the condition number on solution accuracy. If you're studying the same topics, perhaps this will save you some time.